

Hooks

Run commands at specific points in Claude Code's lifecycle. Unlike everything else, hooks are deterministic - they always run. Use them to format, log, notify, or block, guaranteed.

Deterministic · settings.json · Events, matchers & handlers

IN THIS GUIDE

From the first auto-format hook to the full configuration reference:

- Why hooks are **deterministic** - and the everyday use cases
- Configuring them in `settings.json`, the **events** you can hook, and the `/hooks` command
- **Blocking** tool calls with `PreToolUse`, and **sharing** hooks with your team
- The full **lifecycle**, a step-by-step resolve walkthrough, and **hook locations**
- **Matcher patterns** and the five **hook handler types**

ON THIS PAGE

- › 1. Why Use Hooks
- › 2. How Hooks Work
- › 3. A Practical Example: Auto-format
- › 4. Blocking with `PreToolUse`
- › 5. Sharing Hooks with Your Team
- › 6. The Hook Lifecycle
- › 7. How a Hook Resolves
- › 8. Configuration & Hook Locations
- › 9. Matcher Patterns
- › 10. Hook Handler Types

01 Why Use Hooks

You can tell Claude in your `CLAUDE.md` to run Prettier after every file edit. Most of the time it will - but sometimes it won't. A hook makes it happen **every single time**, no exceptions. That determinism is the whole point.

Common use cases include:

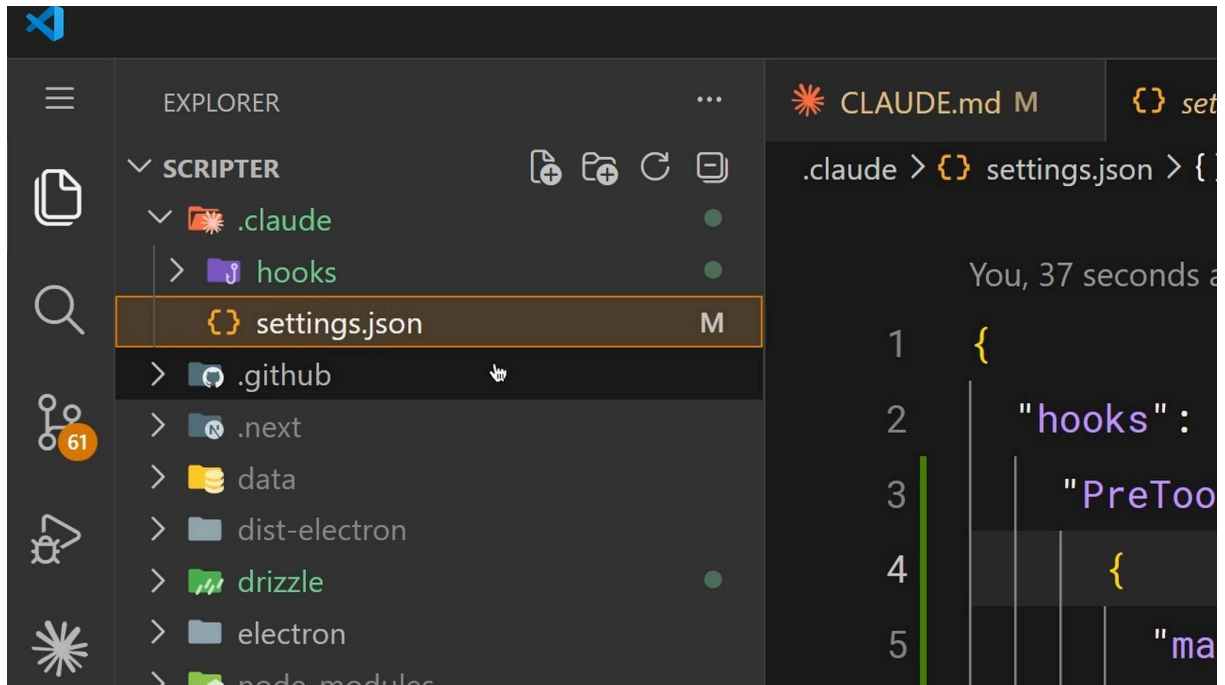
- Auto-formatting after file edits
- Logging all executed commands for compliance
- Blocking dangerous operations like modifying production files
- Sending yourself notifications when Claude finishes a task

02 How Hooks Work

Hooks are configured in your `settings.json`. You pick an **event**, optionally set a **matcher** for which tools it applies to, and provide a **command** to run. The everyday events are:

Event	When it fires
<code>PreToolUse</code>	Before a tool call - can block it.
<code>PostToolUse</code>	After a tool call completes.
<code>UserPromptSubmit</code>	When you submit a prompt, before Claude processes it.
<code>Stop</code>	When Claude finishes responding.
<code>Notification</code>	When Claude sends a notification.

Configure them through the `/hooks` command inside Claude Code, or by editing `settings.json` directly.



Configure hooks with the `/hooks` command inside Claude Code - or edit `settings.json` directly.

03 A Practical Example: Auto-format

The most common hook is auto-formatting after edits. Set a `PostToolUse` hook with a matcher of `Edit|MultiEdit|Write` so it fires whenever Claude modifies a file. The command checks the file extension and runs the appropriate formatter - Prettier for TypeScript, `gofmt` for Go, whatever your project uses.

SETTINGS.JSON

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          { "type": "command", "command": "/path/to/format.sh" }
        ]
      }
    ]
  }
}
```

```
}  
}
```

04 Blocking with PreToolUse

PreToolUse hooks can **block tool calls before they execute**. Your hook receives the tool name and input as JSON on stdin, and its exit code determines the behavior:

Exit code	Behavior
0	Proceed normally.
2	Block the action. The stderr message is fed back to Claude as feedback, so it knows why and can adjust.
Any other	A non-blocking error - shown to you, but nothing stops.

This is how you enforce hard rules: block writes to a production config directory, block bash commands containing `rm -rf`, block commits to `main` - whatever your team needs to be *guaranteed*, not suggested.

```
.claude > settings.json > {} hooks > {} PreToolUse > {} 0  
  
You, 1 minute ago | 1 author (You)  
M  
1 {  
2   "hooks": {  
3     "PreToolUse": [  
4       {  
5         "matcher": "Bash",  
6         "hooks": [  
7           {  
8             "type": "command",  
9             "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/block-dangerous-commands.sh"  
10          }  
11        ]  
12      }  
13    ],  
14    "PostToolUse": [  
15      {  
16        "matcher": "EditWrite",
```

A PreToolUse hook blocking a tool call before it runs - guaranteed, not suggested.

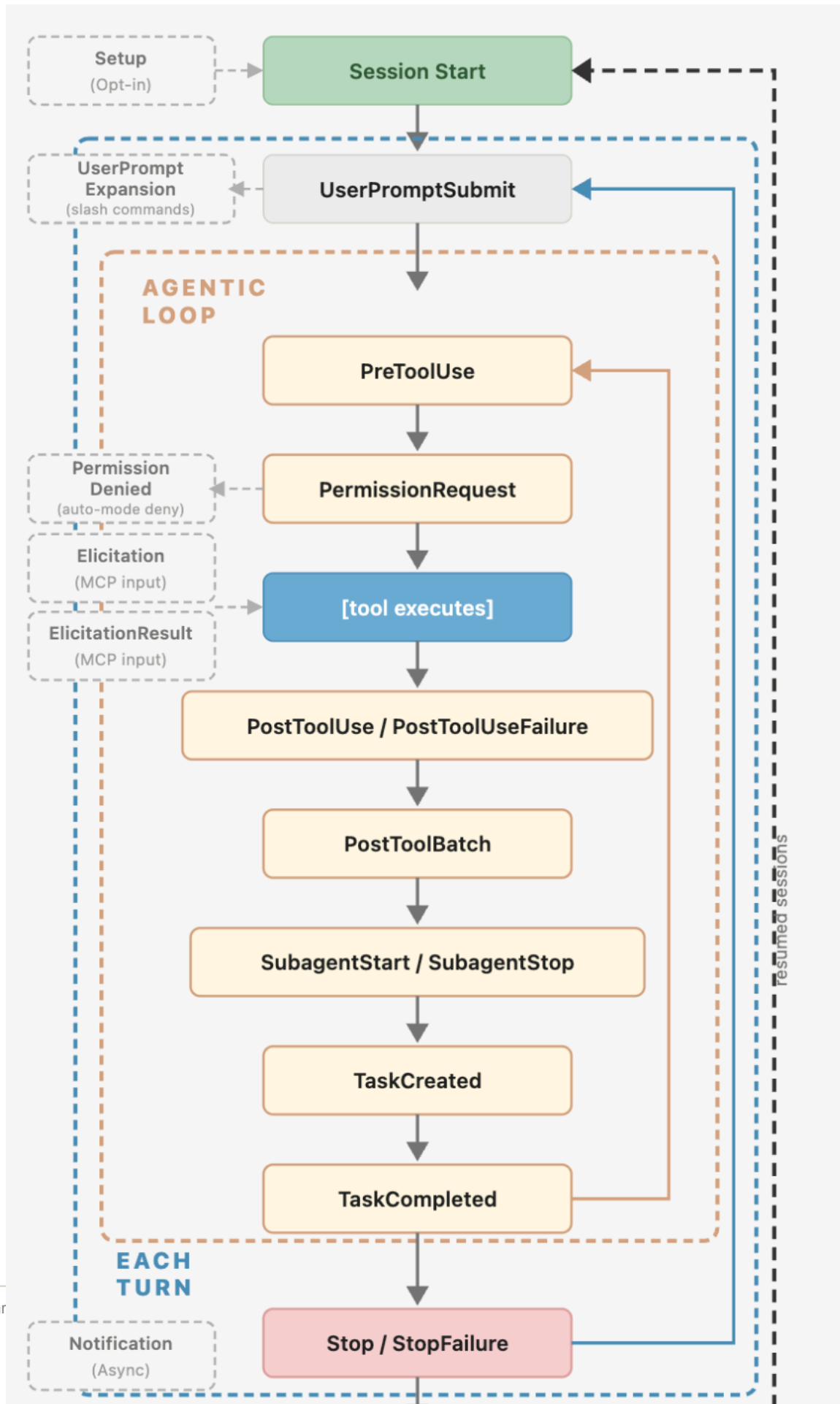
05 Sharing Hooks with Your Team

Hooks configured in `.claude/settings.json` are project-level and can be checked into your repo, so your entire team gets the same hooks automatically. Use the `CLAUDE_PROJECT_DIR` environment variable in your commands to reference scripts stored in your project, so they work regardless of Claude's current working directory.

06 The Hook Lifecycle

Hooks fire at specific points during a session. When an event fires and a matcher matches, Claude Code passes JSON context to your handler - on **stdin** for command hooks, or as the POST body for HTTP hooks. Your handler inspects the input, takes action, and optionally returns a decision. Events fall into three cadences:

- **Once per session** - `SessionStart`, `SessionEnd`.
- **Once per turn** - `UserPromptSubmit`, `Stop`, `StopFailure`.
- **Every tool call** in the agentic loop - `PreToolUse`, `PostToolUse`.



Complete event reference

Every event you can hook, and when it fires:

Event	When it fires
<code>SessionStart</code>	A session begins or resumes.
<code>Setup</code>	Starting with <code>--init-only</code> , or <code>--init / --maintenance</code> in <code>-p</code> mode - for one-time prep in CI or scripts.
<code>UserPromptSubmit</code>	You submit a prompt, before Claude processes it.
<code>UserPromptExpansion</code>	A typed command expands into a prompt, before it reaches Claude. Can block the expansion.
<code>PreToolUse</code>	Before a tool call executes. Can block it.
<code>PermissionRequest</code>	A permission dialog appears.
<code>PermissionDenied</code>	A tool call is denied by the auto-mode classifier. Return <code>{retry: true}</code> to allow a retry.
<code>PostToolUse</code>	After a tool call succeeds.
<code>PostToolUseFailure</code>	After a tool call fails.
<code>PostToolBatch</code>	After a batch of parallel tool calls resolves, before the next model call.
<code>Notification</code>	Claude Code sends a notification.
<code>MessageDisplay</code>	While assistant message text is displayed.
<code>SubagentStart</code>	A subagent is spawned.
<code>SubagentStop</code>	A subagent finishes.
<code>TaskCreated</code>	A task is being created via <code>TaskCreate</code> .
<code>TaskCompleted</code>	A task is being marked completed.
<code>Stop</code>	Claude finishes responding.
<code>StopFailure</code>	The turn ends due to an API error. Output and exit code are ignored.
<code>TeammateIdle</code>	An agent-team teammate is about to go idle.
<code>InstructionsLoaded</code>	A <code>CLAUDE.md</code> or <code>.claude/rules/*.md</code> file loads into context (session start and lazy loads).
<code>ConfigChange</code>	A configuration file changes during a session.

Event	When it fires
CwdChanged	The working directory changes (e.g. a cd command). Useful with direnv.
FileChanged	A watched file changes on disk. The matcher specifies which filenames to watch.
WorktreeCreate	A worktree is being created via --worktree or isolation: "worktree". Replaces default git behavior.
WorktreeRemove	A worktree is being removed (session exit or subagent finish).
PreCompact	Before context compaction.
PostCompact	After context compaction completes.
Elicitation	An MCP server requests user input during a tool call.
ElicitationResult	After a user responds to an MCP elicitation, before the response is sent back.
SessionEnd	A session terminates.

07 How a Hook Resolves

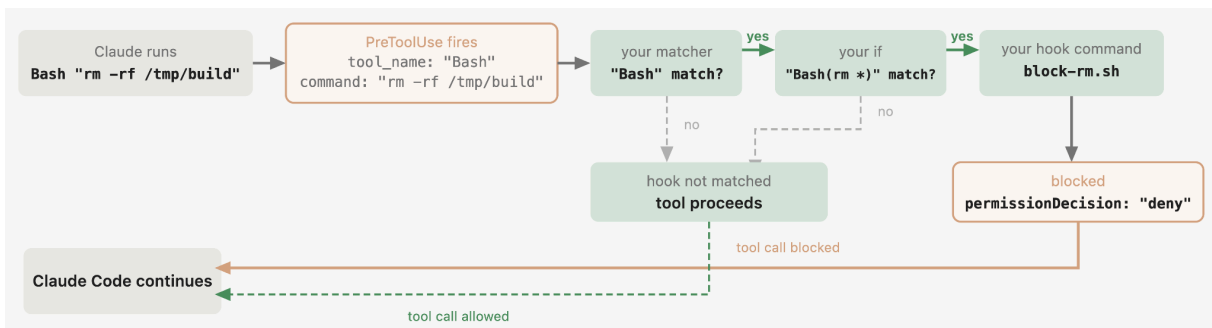
Consider a `PreToolUse` hook that blocks destructive shell commands. The **matcher** narrows to Bash tool calls, and the `if` condition narrows further to subcommands matching `rm *`, so `block-rm.sh` only spawns when both filters match:

```
SETTINGS.JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "if": "Bash(rm *)",
        "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/block-rm.sh",
        "args": []
      }]
    }]
  }
}
```


The script reads the JSON from stdin, extracts the command, and returns a `permissionDecision` of `"deny"` if it contains `rm -rf`:

```
BLOCK-RM.SH
#!/bin/bash
# .claude/hooks/block-rm.sh
COMMAND=$(jq -r '.tool_input.command')
if echo "$COMMAND" | grep -q 'rm -rf'; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "Destructive command blocked by hook"
    }
  }'
else
  exit 0 # no decision; normal permission flow applies
fi
```

Now suppose Claude decides to run `Bash "rm -rf /tmp/build"`. Here's what happens, step by step:



- **1 · Event fires.** The `PreToolUse` event fires; Claude Code sends the tool input as JSON on stdin: `{ "tool_name": "Bash", "tool_input": { "command": "rm -rf /tmp/build" } }`.
- **2 · Matcher checks.** The matcher `"Bash"` matches the tool name, so the group activates. Omitting the matcher (or `"*"`) activates on every occurrence.

- **3 · If-condition checks.** The `if "Bash (rm *)"` matches, so the handler spawns. Had the command been `npm test`, the check would fail and the script would never run - avoiding the spawn. The `if` field is optional.
- **4 · Handler runs.** The script finds `rm -rf` and prints a `"deny"` decision to stdout. A safer `rm file.txt` would hit `exit 0` instead - no decision, so the call continues through the normal permission flow. The hook can deny, but silence doesn't approve.
- **5 · Claude Code acts.** It reads the JSON decision, blocks the tool call, and shows Claude the reason.

08 Configuration & Hook Locations

Hooks are defined in JSON settings files, with three levels of nesting:

- Choose a **hook event** to respond to, like `PreToolUse` or `Stop`.
- Add a **matcher group** to filter when it fires, like “only for the Bash tool.”
- Define one or more **hook handlers** to run when matched.

Throughout, **hook event** is the lifecycle point, **matcher group** is the filter, and **hook handler** is the shell command, HTTP endpoint, MCP tool, prompt, or agent that runs. “Hook” on its own refers to the general feature.

Where you define a hook sets its scope

Location	Scope	Shareable
[object Object]	All your projects	No - local to your machine
[object Object]	Single project	Yes - commit to the repo
[object Object]	Single project	No - gitignored by default
Managed policy settings	Organization-wide	Yes - admin-controlled
Plugin <code>hooks/hooks.json</code>	When plugin is enabled	Yes - bundled with the plugin
Skill / agent frontmatter	While the component is active	Yes - defined in the component

Enterprise administrators can use `allowManagedHooksOnly` to block user, project, and plugin

hooks; hooks from plugins force-enabled in managed `enabledPlugins` are exempt, so admins can distribute vetted hooks through an organization marketplace.

09 Matcher Patterns

The `matcher` field filters when hooks fire. How it's evaluated depends on the characters it contains:

Matcher value	Evaluated as	Example
<code>"*"</code> , <code>" "</code> , or omitted	Match all	Fires on every occurrence of the event
Only letters, digits, <code>_</code> , <code> </code>	Exact string, or <code> </code> -separated list	<code>Bash</code> matches only Bash; <code>Edit Write</code> matches either
Any other character	JavaScript regular expression	<code>^Notebook</code> matches tools starting with Notebook

Each event type matches on a different field. For tool events that field is `tool_name`:

Event(s)	Matcher filters	Example values
PreToolUse, PostToolUse, PostToolUseFailure, PermissionRequest, PermissionDenied	Tool name	<code>Bash</code> , <code>Edit Write</code> , <code>mcp_.*</code>
SessionStart	How the session started	startup, resume, clear, compact
Setup	Which CLI flag triggered setup	init, maintenance
SessionEnd	Why the session ended	clear, resume, logout, ...
Notification	Notification type	permission_prompt, idle_prompt, ...
SubagentStart / SubagentStop	Agent type	general-purpose, Explore, Plan, custom
PreCompact / PostCompact	What triggered compaction	manual, auto
ConfigChange	Configuration source	user_settings, project_settings, ...
FileChanged	Literal filenames to watch	<code>.envrc</code> <code>.env</code>

Event(s)	Matcher filters	Example values
<code>StopFailure</code>	Error type	rate_limit, overloaded, ...
<code>InstructionsLoaded</code>	Load reason	session_start, include, compact, ...
<code>UserPromptExpansion</code>	Command name	Your skill / command names
<code>Elicitation</code> / <code>ElicitationResult</code>	MCP server name	Your configured server names
UserPromptSubmit, PostToolBatch, Stop, CwdChanged, and others	No matcher support	Always fires on every occurrence

Filtering further with if

For tool events you can filter more narrowly with the `if` field on a handler. It uses permission-rule syntax to match the tool name and arguments together: `"Bash(git *)"` runs when any subcommand matches `git *`, and `"Edit(*.ts)"` runs only for TypeScript files.

Matching MCP tools

MCP tools appear as regular tools and follow the pattern `mcp__<server>__<tool>`. To match every tool from a server, append `.*` - it's required, since `mcp__memory` (letters and underscores only) is compared as an exact string and matches nothing. So `mcp__memory__.*` matches all memory-server tools, and `mcp__.*__write.*` matches any `write` tool from any server.

10 Hook Handler Types

Each object in the inner `hooks` array is a **hook handler** - the thing that runs when the matcher matches. There are five types:

- `command` - runs a shell command. Your script gets the JSON input on stdin and replies via exit codes and stdout.
- `http` - sends the JSON input as an HTTP POST; the response body uses the same output format.
- `mcp_tool` - calls a tool on an already-connected MCP server; its text output is treated like command stdout.
- `prompt` - sends a prompt to a Claude model for a single-turn yes/no decision.

- `agent` - spawns a subagent that can use tools like Read, Grep, and Glob to verify conditions (experimental).

Common fields

These apply to all handler types:

Field	Req?	Description
[object Object]	Yes	"command", "http", "mcp_tool", "prompt", or "agent"
[object Object]	No	Permission-rule filter (tool events only), e.g. <code>"Bash(git *)"</code> . Holds exactly one rule - no <code>&&</code> / <code> </code> .
[object Object]	No	Seconds before canceling. Defaults: 600 (command/http/mcp_tool), 30 (prompt), 60 (agent).
[object Object]	No	Custom spinner message shown while the hook runs.
[object Object]	No	Run once per session then remove. Only honored in skill frontmatter.

Command hooks: exec form vs shell form

A command hook runs in **exec form** when `args` is set - `command` is resolved as an executable and spawned directly, with each `args` element passed as one argument and no shell involved. Use it whenever the hook references a path placeholder. Omit `args` for **shell form**, which passes the string to a shell so you can use pipes, `&&`, and globs.

SETTINGS.JSON

```
// exec form - path passed as one argument, no quoting
{ "type": "command", "command": "node",
  "args": ["${CLAUDE_PLUGIN_ROOT}/scripts/format.js", "--fix"] }

// shell form - needs quoting for paths with spaces
{ "type": "command",
  "command": "node \"${CLAUDE_PLUGIN_ROOT}\"/scripts/format.js --fix" }
```

ON WINDOWS

Exec form needs a real executable (a `.exe`). The `.cmd/.bat` shims that npm and friends install aren't executables - invoke the underlying script with `node` directly, or use shell form.

Type-specific fields

Command hooks add `command` (required), `args`, `async`, `asyncRewake`, and `shell` (`"bash"` or `"powershell"`).

HTTP field	Req?	Description
[object Object]	Yes	URL to send the POST request to.
[object Object]	No	Extra headers; values support <code>\$VAR</code> interpolation for listed vars only.
[object Object]	No	Env-var names that may be interpolated into headers. Required for any interpolation.

HTTP error handling differs: non-2xx responses, connection failures, and timeouts are all **non-blocking**. To block, return a 2xx with a JSON body containing `decision: "block"` or a `permissionDecision: "deny"`.

MCP-tool field	Req?	Description
[object Object]	Yes	Name of an already-connected MCP server (never triggers a connection flow).
[object Object]	Yes	Name of the tool to call on that server.
[object Object]	No	Arguments; string values support <code>\${path}</code> substitution, e.g. <code>"\${tool_input.file_path}"</code>

Prompt and agent hooks add `prompt` (required; use `$ARGUMENTS` for the input JSON) and an optional `model`. All matching hooks run in parallel, and identical handlers are deduplicated automatically.

- Recap

KEY TAKEAWAYS

- Hooks run commands at lifecycle points and are **deterministic** - they always run.
- Configure them in `settings.json`: an event, an optional matcher, and a handler.
- `PreToolUse` can **block** a call (exit 2 / a deny decision); `PostToolUse` reacts after.

- Commit project hooks in `.claude/settings.json` so the whole team gets them.
- Handlers come in five types - `command`, `http`, `mcp_tool`, `prompt`, `agent`.

When it has to happen every time, make it a hook - guaranteed, not suggested.

Hooks in Claude Code

Intercepting and Controlling Tool Calls

Hooks in Claude Code allow you to intercept and control tool calls before or after they execute. This gives you fine-grained control over what Claude can and cannot do in your development environment.

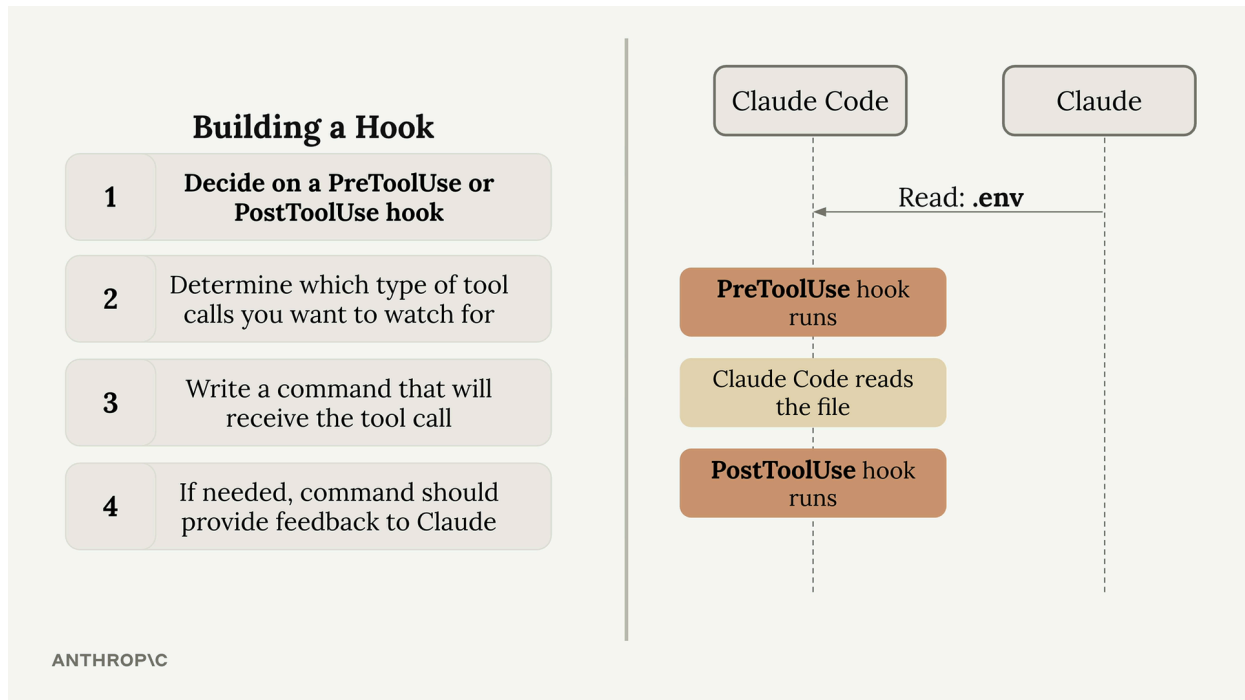
[PreToolUse](#) · [PostToolUse](#) · [Exit codes](#) · [Security](#)

ON THIS PAGE

- › 1. Building a Hook
- › 2. Available Tools
- › 3. Tool Call Data Structure
- › 4. Exit Codes and Control Flow
- › 5. Example Use Case
- › 6. Building the .env Read Hook
- › 7. TypeScript Type Checking Hook
- › 8. Query Duplication Prevention Hook
- › 9. Implementation Considerations
- › 10. Extending These Concepts
- › 11. Other Hook Types

01 Building a Hook

Creating a hook involves four main steps:



- **Decide on a PreToolUse or PostToolUse hook** - PreToolUse hooks can prevent tool calls from executing, while PostToolUse hooks run after the tool has already been used.
- **Determine which type of tool calls you want to watch for** - you need to specify exactly which tools should trigger your hook.
- **Write a command that will receive the tool call** - this command gets JSON data about the proposed tool call via standard input.
- **If needed, the command should provide feedback to Claude** - your command's exit code tells Claude whether to allow or block the operation.

02 Available Tools

Claude Code provides several built-in tools that you can monitor with hooks:

Building a Hook

- 1 Decide on a PreToolUse or PostToolUse hook
- 2 **Determine which type of tool calls you want to watch for**
- 3 Write a command that will receive the tool call
- 4 If needed, command should provide feedback to Claude

ANTHROPIC

Tool Names

Tool Name	Purpose
Read	Read a file
Edit, MultiEdit	Edit an existing file
Write	Create a file and write to it
Bash	Execute a command
Glob	Find files/folders based upon a pattern
Grep	Search for content
Task	Create a sub-agent to complete a particular task
WebFetch, WebSearch	Search or fetch a particular page

To see exactly which tools are available in your current setup, you can ask Claude directly for a list. This is especially useful since the available tools can change when you add custom MCP servers.

03 Tool Call Data Structure

When your hook command executes, Claude sends JSON data through standard input containing details about the proposed tool call:

Building a Hook

- 1 Decide on a PreToolUse or PostToolUse hook
- 2 Determine which type of tool calls you want to watch for
- 3 **Write a command that will receive the tool call**
- 4 If needed, command should provide feedback to Claude

ANTHROPIC

Tool Call Data

```
{
  "session_id": "2d6a1e4d-6...",
  "transcript_path": "/Users/sg/...",
  "hook_event_name": "PreToolUse",
  "tool_name": "Read",
  "tool_input": {
    "file_path": "/code/queries/.env"
  }
}
```

Standard In

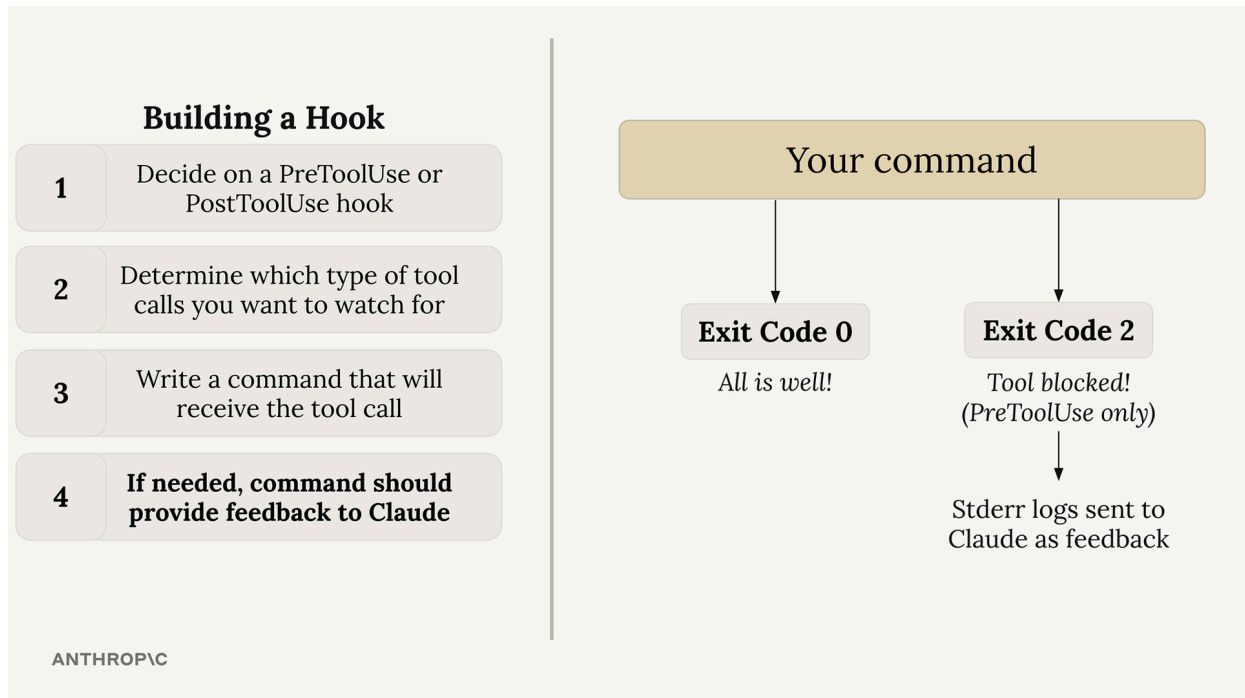
Your command

```
{
  "session_id": "2d6a1e4d-6...",
  "transcript_path": "/Users/sg/...",
  "hook_event_name": "PreToolUse",
  "tool_name": "Read",
  "tool_input": {
    "file_path": "/code/queries/.env"
  }
}
```

Your command reads this JSON from standard input, parses it, and then decides whether to allow or block the operation based on the tool name and input parameters.

04 Exit Codes and Control Flow

Your hook command communicates back to Claude through exit codes:



- **Exit Code 0** - everything is fine, allow the tool call to proceed.
- **Exit Code 2** - block the tool call (PreToolUse hooks only).

When you exit with code 2 in a PreToolUse hook, any error messages you write to standard error will be sent to Claude as feedback, explaining why the operation was blocked.

05 Example Use Case

A common use case is preventing Claude from reading sensitive files like `.env` files. Since both the `Read` and `Grep` tools can access file contents, you'd want to monitor both tool types and check if they're trying to access restricted file paths.

This approach gives you complete control over Claude's file system access while providing clear feedback about why certain operations are restricted.

Let's build a hook that prevents Claude from reading sensitive files. This is a practical example of how a PreToolUse hook can intercept tool calls before they run.

06 Building the `.env` Read Hook

6.1 What This Exercise Covers

You'll write a hook that blocks the `Read` tool from opening `.env`. This protects your environment variables during a session.

SCOPE

This hook covers `Read` only. Blocking `Grep` or `Bash` from reaching the same file requires checking each tool's input shape separately, since each tool sends different fields. For comprehensive file protection, combine a hook with a `permissions.deny` rule like `"Read (**/.env) "`.

6.2 Configure the Hook

Open `.claude/settings.local.json` and add a `PreToolUse` hook that matches the `Read` tool:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          { "type": "command", "command": "node $PWD/hooks/read_hook.js" }
        ]
      }
    ]
  }
}
```

6.3 Write the Hook Script

Create `hooks/read_hook.js`:

```
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (d) => (input += d));
process.stdin.on("end", () => {
  const toolArgs = JSON.parse(input);
  const readPath = toolArgs.tool_input?.file_path || "";
  if (readPath.includes(".env")) {
```

```
    console.error("You cannot read the .env file");  
    process.exit(2);  
  }  
  process.exit(0);  
});
```

The hook reads the tool call from stdin as JSON, checks `tool_input.file_path`, and exits with code 2 to block the call (anything written to stderr becomes the message Claude sees).

6.4 Test It

In a Claude Code session, ask Claude to read your `.env` file. You should see:

```
You cannot read the .env file
```

That's the hook blocking the `Read` call. Ask Claude to read a different file and it works normally.

6.5 Why Read Only?

Each tool sends a different input shape. `Read` sends `{"file_path": "..."};` `Grep` sends `{"pattern": "...", "path": "..."};` where `path` is a search directory, not a file; `Bash` sends `{"command": "..."};` A check on `file_path` catches `Read` but won't catch a project-wide grep for `API_KEY` or a `cat .env` in Bash. To cover those, you'd write separate matchers per tool and inspect each one's specific fields - or use `permissions.deny` rules, which apply uniformly across tools.

6.6 Sharing Settings Across Machines

You may notice that after running the `npm run setup` command there are two `settings.json` files in the `.claude` directory. Let me explain what's going on there.

The Claude Code documentation lists some recommendations around hooks security:

Security Best Practices

Here are some key practices for writing more secure hooks:

1. **Validate and sanitize inputs** - Never trust input data blindly
2. **Always quote shell variables** - Use `"$VAR"` not `$VAR`
3. **Block path traversal** - Check for `..` in file paths
4. **Use absolute paths** - Specify full paths for scripts
5. **Skip sensitive files** - Avoid `.env` , `.git/` , keys, etc.

One of the recommendations is to use absolute paths (rather than relative paths) for scripts. This helps mitigate path interception and binary planting attacks.

This recommendation also makes it much more challenging to share `settings.json` files. The reason is simple: the absolute path to any of the hook scripts on your machine will likely be different from the absolute path on my machine, simply because we will probably place the project in separate directories.

To solve this problem, our project has a `settings.example.json` file. Inside of it, the script references contain a `$PWD` placeholder. When we run `npm run setup`, some dependencies are installed, but it also runs an `init-claude.js` script placed inside the scripts directory. This script will replace those `$PWD` placeholders with the absolute path to the project on your machine, copy the `settings.example.json` file, and rename it to `settings.local.json`.

This script allows us to share `settings.json` files but still use the recommended absolute paths.

07 TypeScript Type Checking Hook

One of the most useful hooks addresses a fundamental problem: when Claude modifies a function signature, it often doesn't update all the places where that function is called throughout your project.

For example, if you ask Claude to add a `verbose` parameter to a function in `schema.ts`, it will successfully update the function definition but miss the call site in `main.ts`. This creates type errors that Claude doesn't immediately catch.

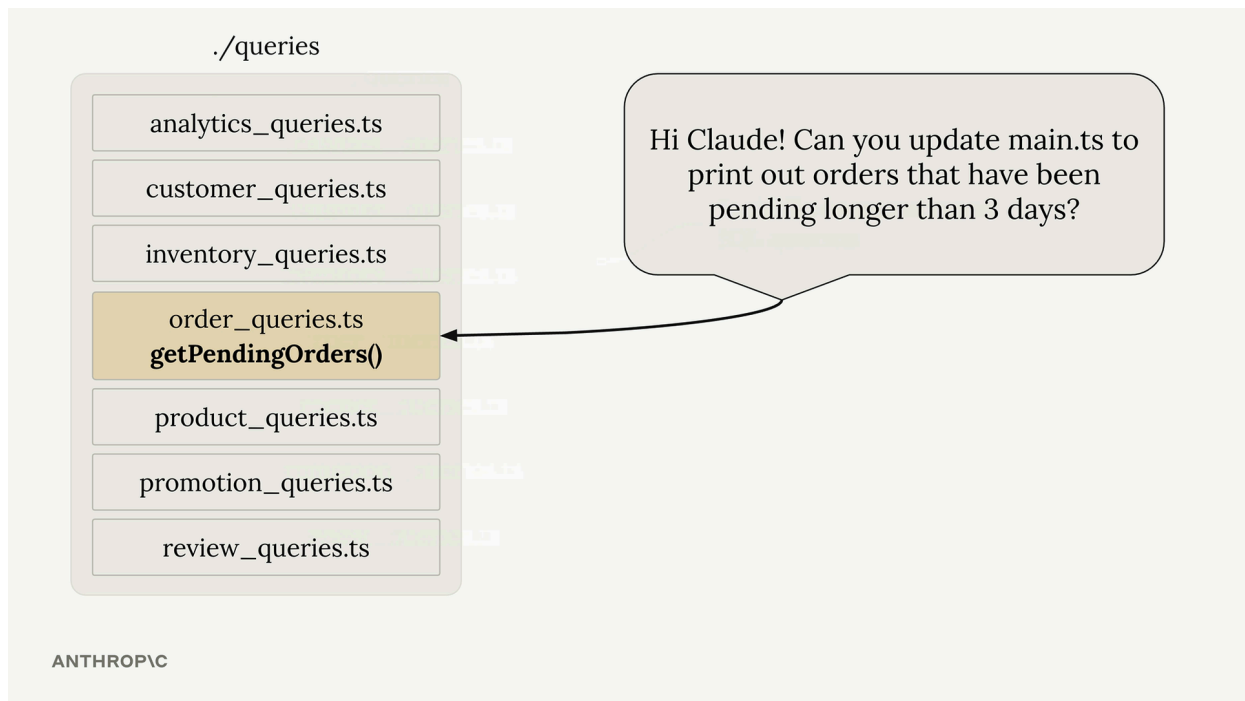
The solution is a post-tool-use hook that runs the TypeScript compiler after every file edit:

- Runs `tsc --noEmit` to check for type errors.
- Captures any errors found.
- Feeds the errors back to Claude immediately.
- Prompts Claude to fix the issues in other files.

This hook works for any typed language where you can run a type checker. For untyped languages, you could implement similar functionality using automated tests instead.

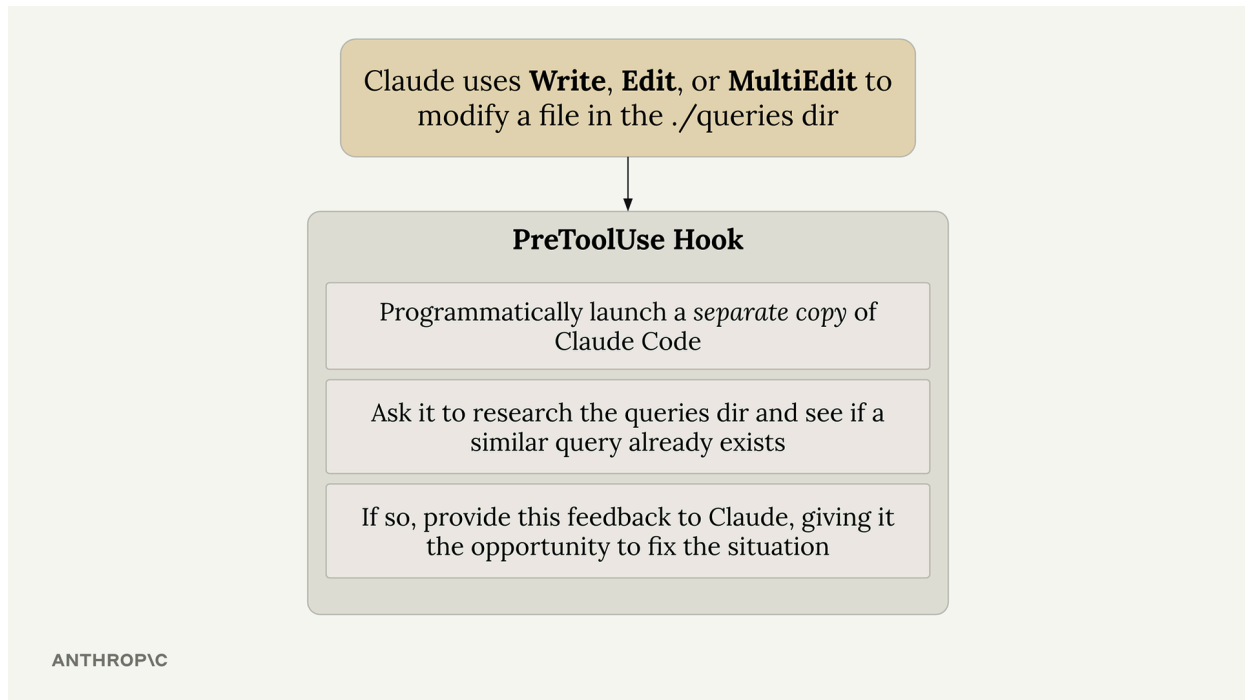
08 Query Duplication Prevention Hook

In larger projects with many database queries, Claude sometimes creates duplicate functionality instead of reusing existing code. This is especially problematic when you give Claude complex, multi-step tasks that include database operations as just one component.



Consider a project structure with multiple query files, each containing many SQL functions. When you ask Claude to “create a Slack integration that alerts about orders pending longer than 3 days,” it might write a new query instead of using the existing `getPendingOrders()` function.

The query duplication hook addresses this by implementing a review process:



Here's how it works:

- Triggers when Claude modifies files in the `./queries` directory.
- Launches a separate instance of Claude Code programmatically.
- Asks the second instance to review the changes and check for similar existing queries.
- If duplicates are found, provides feedback to the original Claude instance.
- Prompts Claude to remove the duplicate and use the existing functionality.

09 Implementation Considerations

Both hooks use the pre-tool-use or post-tool-use hook system. The TypeScript hook is relatively lightweight and runs quickly. The query duplication hook requires more resources since it launches a separate Claude instance for each review.

For the query hook, consider these trade-offs:

- **Benefits** - a cleaner codebase with less duplication.
- **Costs** - additional time and API usage for each query directory edit.
- **Recommendation** - only monitor critical directories to minimize overhead.

The hooks use Claude's Agent SDK to programmatically interact with the AI. This allows you to create sophisticated workflows where one Claude instance can review and provide feedback on another's work.

10 Extending These Concepts

These hooks demonstrate broader principles you can apply to your own projects:

- Use compiler/linter output to provide immediate feedback.
- Implement code review processes using separate AI instances.
- Focus monitoring on high-value directories where consistency matters most.
- Balance automation benefits against performance costs.

The key is identifying the specific pain points in your development workflow and creating targeted hooks that address those issues automatically.

11 Other Hook Types

There are more hooks beyond the PreToolUse and PostToolUse hooks discussed in this course. There are also:

- **Notification** - runs when Claude Code sends a notification, which occurs when Claude needs permission to use a tool, or after Claude Code has been idle for 60 seconds.
- **Stop** - runs when Claude Code has finished responding.
- **SubagentStop** - runs when a subagent (displayed as a "Task" in the UI) has finished.
- **PreCompact** - runs before a compact operation occurs, either manual or automatic.
- **UserPromptSubmit** - runs when the user submits a prompt, before Claude processes it.
- **SessionStart** - runs when starting or resuming a session.
- **SessionEnd** - runs when a session ends.

THE CONFUSING PART

The stdin input to your commands will change based on the type of hook being executed (PreToolUse, PostToolUse, Notification, etc.).

The `tool_input` contained in that will differ based on the tool that was called (in the case of `PreToolUse` and `PostToolUse` hooks).

For example, here's a sample of some stdin input to a hook, where the hook is a `PostToolUse` that was watching for uses of the `TodoWrite` tool. For reference, that is the tool that Claude uses to keep track of to-do items.

```
{
  "session_id": "9ecf22fa-edf8-4332-ae85-b6d5456eda64",
  "transcript_path": "<path_to_transcript>",
  "hook_event_name": "PostToolUse",
  "tool_name": "TodoWrite",
  "tool_input": {
    "todos": [{ "content": "write a readme", "status": "pending", "id": "1" }]
  },
  "tool_response": {
    "oldTodos": [],
    "newTodos": [{ "content": "write a readme", "status": "pending", "id": "1" }]
  }
}
```

And for comparison, here's an example of the input to a `Stop` hook:

```
{
  "session_id": "af9f50b6-f042-4773-b3e2-c3a4814765ce",
  "transcript_path": "<path_to_transcript>",
  "hook_event_name": "Stop",
  "stop_hook_active": false
}
```

As you can see, the stdin input to your command will differ significantly based on the hook (`PreToolUse`, `PostToolUse`, `Stop`, etc.) and the matcher used. This can make writing hooks challenging - you might not know the exact structure of the input to your command.

To handle this challenge, try making a helper hook like this:

```
"PostToolUse": [ // Or "PreToolUse" or "Stop", etc
{
  "matcher": "*",
  "hooks": [
```

```
{
  "type": "command",
  "command": "jq . > post-log.json"
}
],
]
```

Notice the provided command. It will write the input to this hook to the `post-log.json` file, which lets you inspect exactly what would have been fed into your command. This makes it a lot easier to understand what data your command should inspect.

Hooks turn Claude Code into a programmable, guard-railed teammate - intercept what matters, and let everything else run.